Лабораторная работа №3

«Решение задач многомерной условной оптимизации методом множителей Лагранжа»

Выполнил: Рожновский Юрий, K3339, п. 1.2

Преподаватель: Духанов А. В.

Цель работы

Изучить метод множителей Лагранжа для решения задач многомерной условной оптимизации при ограничениях-равенствах; научиться проверять условия применимости метода, строить функцию Лагранжа, составлять и решать систему стационарности, находить все точки-кандидаты, проверять их допустимость и обосновывать глобальный характер найденных экстремумов.

Краткие теоретические сведения

Условная оптимизация — это поиск наибольшего или наименьшего значения целевой функции при дополнительных ограничениях, которым должны удовлетворять переменные.

В задаче без ограничений внутренние стационарные точки гладкой функции находятся из условия равенства её градиента нулю. При наличии ограничений переменные нельзя изменять произвольно: они должны оставаться в пределах, заданных ограничениями.

Геометрический смысл метода множителей Лагранжа состоит в том, что в регулярной точке условного экстремума градиент целевой функции перпендикулярен допустимой поверхности. Поэтому градиент ограничения и градиент целевой функции должны быть параллельны. Для нескольких ограничений градиент целевой функции выражается как линейная комбинация градиентов ограничений:

$$\nabla f(x) + \lambda \nabla g(x) = 0$$

Здесь λ — множитель Лагранжа, то есть дополнительная неизвестная величина. Для задачи с несколькими ограничениями вводится отдельный множитель для каждого равенства.

Для одного ограничения $g(x, y) = 0$ функцию Лагранжа можно записать в виде:

$$L(x, y, \lambda) = f(x, y) + \lambda g(x, y)$$

Точки-кандидаты находят из системы, состоящей из равенств частных производных функции Лагранжа по исходным переменным нулю и исходных ограничений, т. к. производная по λ воспроизводит ограничение.

Метод Лагранжа формирует необходимые условия условного локального экстремума в точке, но не гарантирует, что найденная точка является минимумом или максимумом. После решения системы необходимо проверить допустимость кандидатов, классифицировать их и отдельно обосновать глобальность.

Доказать глобальность экстремумов можно, например, через компактность и непрерывность или выпуклость.

Постановка задачи для варианта 4

Для варианта 4 требуется найти глобальные минимум и максимум целевой функции при заданном ограничении.

Целевая функция:

$$f(x, y) = x^2 + 2y^2$$

Ограничение:

$$x^2 + y^2 = 1$$

Перенесём все слагаемые ограничения в левую часть и введём обозначение:

$$g(x, y) = x^2 + y^2 - 1 = 0$$

Переменные задачи – x и y. Целевая функция определена на всей плоскости R^2. Функция ограничения также определена на всей плоскости. Допустимое множество – единичная окружность с центром в начале координат.

Анализ задачи и применимости метода

1) Проверка области определения и гладкости

Функция $f(x, y) = x^2 + 2y^2$ и функция $g(x, y) = x^2 + y^2 - 1$ являются многочленами. Они определены при любых действительных x и y и имеют непрерывные частные производные любого порядка. Следовательно, требование непрерывной дифференцируемости целевой функции и ограничения выполнено.

Частные производные целевой функции:

$$\partial f/\partial x = 2x; \quad \partial f/\partial y = 4y$$

Частные производные функции ограничения:

$$\partial g/\partial x = 2x; \quad \partial g/\partial y = 2y$$

2) Проверка непустоты допустимого множества

Допустимое множество задаётся уравнением $x^2 + y^2 = 1$. Проверим точку (1, 0): $1^2 + 0^2 = 1$. Равенство выполняется, следовательно, существует хотя бы одна допустимая точка и допустимое множество непусто.

3) Проверка регулярности ограничения

Для проверки регулярности необходимо определить ранг матрицы Якоби ограничений. В допустимых точках ранг матрицы Якоби должен быть равен количеству ограничений.

Вычислим частные производные ограничения по переменным x и y:

$$\partial g/\partial x = 2x; \partial g/\partial y = 2y$$

Составим матрицу Якоби:

$$Jg(x, y) = (2x\ 2y)$$

Матрица Якоби имеет одну строку и два столбца. Её ранг равен нулю только в том случае, если оба элемента строки одновременно равны нулю:

$$2x = 0; 2y = 0$$

Отсюда получаем $x = 0$ и $y = 0$. Однако точка (0, 0) не принадлежит допустимому множеству, поскольку не удовлетворяет исходному ограничению:

$$0^2 + 0^2 \neq 1$$

Следовательно, во всех допустимых точках ранг матрицы Якоби равен единице, то есть количеству ограничений.

4) Вывод о применимости метода

Целевая функция и ограничение непрерывно дифференцируемы; допустимое множество непусто; ограничение является регулярным. Следовательно, все необходимые условия для применения метода множителей Лагранжа выполнены.

Исследование существования глобальных экстремумов

Допустимое множество задаётся уравнением $x^2 + y^2 = 1$ и представляет собой единичную окружность. Оно замкнуто, поскольку является множеством уровня непрерывной функции $x^2 + y^2$, и ограничено, поскольку для любой допустимой точки выполняются неравенства $-1 \leq x \leq 1$ и $-1 \leq y \leq 1$.

В конечномерном пространстве замкнутое и ограниченное множество компактно. Целевая функция $f(x, y) = x^2 + 2y^2$ непрерывна на всей плоскости, а значит, и на допустимом множестве.

По теореме Вейерштрасса непрерывная функция на компактном множестве достигает на нём глобального минимума и глобального максимума. Следовательно, в данной задаче оба глобальных экстремума существуют. После нахождения всех точек-кандидатов достаточно сравнить значения целевой функции в них.

Построение функции Лагранжа

Для одного ограничения-равенства функция Лагранжа записывается как сумма целевой функции и ограничения, умноженного на дополнительную неизвестную λ:

$$L(x, y, \lambda) = f(x, y) + \lambda g(x, y)$$

Подставим целевую функцию и ограничение:

$$L(x, y, \lambda) = x^2 + 2y^2 + \lambda(x^2 + y^2 - 1)$$

Система стационарности

Найдём частные производные функции Лагранжа по всем неизвестным x, y и λ.

Производная по x:

$$\partial L/\partial x = 2x + 2\lambda x = 2x(1 + \lambda)$$

Производная по y:

$$\partial L/\partial y = 4y + 2\lambda y = 2y(2 + \lambda)$$

Производная по λ:

$$\partial L/\partial \lambda = x^2 + y^2 - 1$$

Приравняем каждую производную к нулю. Получим систему стационарности:

$$\begin{cases} 2x(1 + \lambda) = 0 \\ 2y(2 + \lambda) = 0 \\ x^2 + y^2 - 1 = 0 \end{cases}$$

Ручное решение системы

Рассмотрим систему:

$$\begin{cases} 2x(1 + \lambda) = 0 \\ 2y(2 + \lambda) = 0 \\ x^2 + y^2 - 1 = 0 \end{cases}$$

Поскольку произведение равно нулю, в каждом из первых двух уравнений хотя бы один множитель должен быть равен нулю. Рассмотрим все возможные случаи.

1) Первый случай: x = 0

Подставим x = 0 в ограничение:

$$0^2 + y^2 = 1 \Rightarrow y^2 = 1 \Rightarrow y = \pm 1$$

Получаем две точки: (0, 1) и (0, −1). Найдём множитель λ из второго уравнения системы:

$$2y(2 + \lambda) = 0$$

В найденных точках y не равен нулю, поэтому 2 + λ = 0 => λ = −2. Получаем два решения системы: (x, y, λ) = (0, 1, −2) и (0, −1, −2).

2) Второй случай: y = 0

Подставим y = 0 в ограничение:

$$x^2 + 0^2 = 1 \Rightarrow x^2 = 1 \Rightarrow x = \pm 1$$

Получаем две точки: (1, 0) и (−1, 0). Найдём множитель λ из первого уравнения:

$$2x(1 + \lambda) = 0$$

В найденных точках x не равен нулю, поэтому 1 + λ = 0, откуда λ = −1. Получаем два решения системы: (1, 0, −1) и (−1, 0, −1).

3) Третий случай: x ≠ 0 и y ≠ 0

Если x и y одновременно не равны нулю, то из первого уравнения следует $1 + \lambda = 0 \Rightarrow \lambda = -1$. Из второго уравнения следует $2 + \lambda = 0 \Rightarrow \lambda = -2$. Одновременно λ не может равняться -1 и -2. Получено противоречие, поэтому решений в этом случае нет.

Таким образом, система имеет четыре действительных решения. Других случаев нет, поскольку для каждого из первых двух уравнений были рассмотрены все варианты обращения произведения в ноль.

№	x	y	λ
1	1	0	−1
2	−1	0	−1
3	0	1	−2
4	0	−1	−2

Проверка найденных кандидатов

Каждую найденную точку необходимо проверить подстановкой в исходное ограничение. Это подтверждает, что точка действительно допустима.

Точка	λ	g(x, y)	f(x, y)	Допустимость
(1, 0)	-1	0	1	Да
(−1, 0)	-1	0	1	Да
(0, 1)	−2	0	2	Да
(0, −1)	−2	0	2	Да

Во всех четырёх случаях функция определена, $g(x, y) = x^2 + y^2 - 1$ равна нулю, поэтому все кандидаты удовлетворяют ограничению.

Значения целевой функции вычисляются по формуле $f(x, y) = x^2 + 2y^2$. В точках (1, 0) и (−1, 0) значение равно 1. В точках (0, 1) и (0, −1) значение равно 2.

Классификация точек и обоснование глобальности

Среди всех найденных кандидатов наименьшее значение целевой функции равно 1 и достигается в точках (1, 0) и (−1, 0). Наибольшее значение равно 2 и достигается в точках (0, 1) и (0, −1).

Поскольку в пункте «Исследование существования глобальных экстремумов» уже было сказано, что глобальные минимум и максимум существуют, а все допустимые точки-кандидаты найдены, сравнение значений функции позволяет определить глобальные экстремумы.

Дополнительно результат можно проверить, подставив ограничение $y^2 = 1 - x^2$ в целевую функцию. Получим:

$$f(x, y) = x^2 + 2y^2 = x^2 + 2(1 - x^2) = 2 - x^2$$

Из ограничения получим $0 \le x^2 \le 1 \Rightarrow 1 \le 2 - x^2 \le 2$. Минимум достигается при $x^2 = 1$, то есть в точках (1, 0) и (−1, 0). Максимум достигается при $x^2 = 0$, то есть в точках (0, 1) и (0, −1). Это непосредственно подтверждает найденные глобальные экстремумы.

Результаты ручного метода

Функция Лагранжа	$L = x^2 + 2y^2 + \lambda(x^2 + y^2 - 1)$
Система стационарности	$\begin{cases} 2x(1 + \lambda) = 0 \\ 2y(2 + \lambda) = 0 \\ x^2 + y^2 - 1 = 0 \end{cases}$
Точки-кандидаты	(1, 0), (−1, 0), (0, 1) и (0, −1)
Глобальный минимум	(1, 0) и (−1, 0)
Глобальный максимум	(0, 1) и (0, −1)

Программное решение

Весь код программы можно найти в файле lagrange_method.py (https://github.com/hamster1812/5_sem_optimization_methods/blob/main/lab_3/lagrange_method.py)

Рассмотрим программное решение.

```
import itertools
import sympy as sp

EPS = 1e-8
```

При решении используются библиотеки itertools и sympy. Также определена допустимая погрешность.

```
def is_real_finite(value):
    value = sp.simplify(value)
    if value.free_symbols or not value.is_real:
        return False
    try:
        z = float(sp.N(value, 25))
        return abs(z) != float("inf") and z == z
    except (TypeError, ValueError, OverflowError):
        return False
```

Функция проверяет, является ли значение фактическим и конечным.

```
def solve_system(equations, unknowns):
    try:
        raw = sp.solve(equations, unknowns, dict=True)
    except (NotImplementedError, ValueError, TypeError, sp.SympifyError) as error:
        print("Предупреждение: система решена не полностью:", error)
        return [], False
    result, complete = [], True
    for item in raw:
        if not all(v in item for v in unknowns):
            complete = False
            continue
        if not all(is_real_finite(item[v]) for v in unknowns):
            complete = False
            continue
        result.append(item)
    return result, complete
```

Функция принимает систему ограничений и неизвестные, относительно которых происходит решение. Дополнительно проверяется,

что для каждой найденной точки определены все неизвестные и их значения фактически и конечны. Возвращает список точек и все ли точки были допущены.

```
def find_singular_points(constraints, variables):
    m = len(constraints)
    if m == 0:
        return [], True
    jac = sp.Matrix(constraints).jacobian(variables)
    minors = [
        jac[:, list(cols)].det()
        for cols in itertools.combinations(range(len(variables)), m)
    ]
    return solve_system(constraints + minors, variables)
```

Функция находит точки, в которых ранг матрицы Якоби меньше m.

```
def choose_candidates(points, function, variables, constraints):
    candidates = []

    for solution, kind in points:
        point = {v: solution[v] for v in variables}
        residuals = [sp.simplify(g.subs(point)) for g in constraints]
        if not all(is_real_finite(r) for r in residuals):
            continue
        if any(abs(float(sp.N(r, 25))) > EPS for r in residuals):
            continue
        value = sp.simplify(function.subs(point))
        if not is_real_finite(value):
            continue
        for old in candidates:
            if all(sp.simplify(old["point"][v] - point[v]) == 0 for v in
variables):
                continue
        candidates.append({
            "point": point, "solution": solution, "value": value,
            "residuals": residuals, "kind": kind
        })

    return candidates
```

Функция отбирает подходящих кандидатов из найденных. Отбор проходит по нескольким критериям: невязки фактически и конечны, а также находятся в пределах погрешности; значение в точке является фактическим и конечным; точка не является дубликатом уже отобранной точки.

```
def classify_candidates(candidates, variables, multipliers, minimum, maximum,
                        global_min_exist=False, global_max_exist=False):
    ext_type = {
        (False, False) : "no",
        (True, False) : "min",
        (False, True) : "max",
        (True, True) : "both"
    }[(global_min_exist, global_max_exist)]

    for i, c in enumerate(candidates, 1):
        classification = "нельзя точно сказать"
        if ext_type == "min":
            if c["value"] == minimum:
                classification = "глобальный минимум"
        elif ext_type == "max":
            if c["value"] == maximum:
                classification = "глобальный максимум"
        elif ext_type == "both":
            if c["value"] == minimum:
                classification = "глобальный минимум"
            if c["value"] == maximum:
                classification = "глобальный максимум"

        c["classification"] = classification
        print_candidate_stats(c, i, variables, multipliers)
```

Функция, при наличии доказательства глобальности экстремумов, классифицирует кандидатов, являются ли они глобальными минимумами или максимумами. Также выводит статистику по кандидату.

```
def solve_lagrange(function, variables, constraints, global_min_exist=False,
global_max_exist=False):
    if not variables or len(set(variables)) != len(variables):
        raise ValueError("Нужен непустой список различных переменных.")
    if len(constraints) >= len(variables):
        raise ValueError("Число ограничений должно быть меньше числа переменных.")

    multipliers = list(sp.symbols(f"lambda1:{len(constraints)+1}", real=True))
    L = function + sum(l * g for l, g in zip(multipliers, constraints))
    derivatives = [sp.diff(L, v) for v in variables]
    equations = derivatives + constraints
    unknowns = variables + multipliers

    print_initial_stats(function, constraints, L, equations)

    regular, regular_ok = solve_system(equations, unknowns)
    singular, singular_ok = find_singular_points(constraints, variables)
    points = (tuple(zip(regular, ["reg"] * len(regular))) +
              tuple(zip(singular, ["sing"] * len(singular))))

    candidates = choose_candidates(points, function, variables, constraints)
    candidates.sort(key=lambda x: list(x["point"].values()))

    minimum = maximum = None
    if global_min_exist:
```

```
        minimum = min([c["value"] for c in candidates])
    if global_max_exist:
        maximum = max([c["value"] for c in candidates])

    classify_candidates(candidates, variables, multipliers, minimum, maximum,
                        global_min_exist=global_min_exist,
                        global_max_exist=global_max_exist)

    result = {
        "candidates": candidates, "regular_complete": regular_ok,
        "singular_complete": singular_ok, "minimum": minimum, "maximum": maximum
    }
    print_stats(result, variables, global_min_exist=global_min_exist,
global_max_exist=global_max_exist)
```

Функция является основной частью программы. Сначала проверяется наличие переменных и число ограничений. Потом создаются переменные для лямбд, функция Лагранжа, производные функции, система стационарности, список неизвестных. Выводятся начальные данные. Находятся точки решения системы и точки с понижением ранга, отбираются допустимые кандидаты, при возможности вычисляется минимум и максимум функции, проводится классификация кандидатов. Дальше формируется результат и выводится итоговая статистика.

```
def print_initial_stats(function, constraints, L, equations):
    print("\nЦелевая функция: f =", function)
    print("Ограничения (нулевая форма):")
    for constraint in constraints:
        print(constraint)
    print("Функция Лагранжа: L =", L)
    print("Уравнения стационарности и ограничения:")
    for equation in equations:
        print(equation, "= 0")
```

Функция вывода начальных данных.

```
def print_candidate_stats(c, k, variables, multipliers):
    point = c["point"]
    print(f"\nКандидат {k}:")
    print("  точка:", [point[v] for v in variables])
    print("  множители:", [c["solution"][lam] for lam in multipliers])
    print("  значение функции:", c["value"])
    print("  невязки ограничений:", c["residuals"])
    print("  получена:", ({"reg": "при решении системы",
                           "sing": "через понижение матрицы Якоби"
                           }[c["kind"]]))
    print("  классификация:", c["classification"])
```

Функция вывода данных кандидата.

```
def print_stats(result, variables, global_min_exist=False,
global_max_exist=False):
    regular_ok = result["regular_complete"]
    singular_ok = result["singular_complete"]
    candidates = result["candidates"]
    all_found = regular_ok and singular_ok

    print("\nИтог:")
    if all_found and candidates:
        minimum = result["minimum"]
        maximum = result["maximum"]
        if global_min_exist:
            min_points = [c for c in candidates if abs(c["value"] - minimum) <=
EPS]
            print("  глобальный минимум:", minimum)
            print("  точки минимума:", [tuple(c["point"][v] for v in variables)
for c in min_points])
            if global_max_exist:
                max_points = [c for c in candidates if abs(c["value"] - maximum)
<= EPS]
                print("  глобальный максимум:", maximum)
                print("  точки максимума:", [tuple(c["point"][v] for v in
variables) for c in max_points])
        elif global_max_exist:
            print("  глобальный минимум не доказан, поэтому он не был
классифицирован.")
            max_points = [c for c in candidates if abs(c["value"] - maximum) <=
EPS]
            print("  глобальный максимум:", maximum)
            print("  точки максимума:", [tuple(c["point"][v] for v in variables)
for c in max_points])
        else:
            print("  так как наличие глобальных экстремумов не доказано, нет
возможности классификации.")
    elif not all_found:
        print("  Решатель вернул неполный результат или параметрическое
семейство.")
    else:
        print("  Не было найдено ни одного кандидата.")

    print("-" * 44)
    print("Количество найденных кандидатов:", len(result["candidates"]))
    print("Система стационарности решена полностью:",
          "да" if result["regular_complete"] else "нет")
    print("Поиск особых точек выполнен полностью:",
          "да" if result["singular_complete"] else "нет")
```

Функция вывода итоговых результатов.

```
def example_variant_4():
    x, y = sp.symbols("x y", real=True)
```

```
    expression = x**2 + 2 * (y + 0)**2
    variables = [x, y]
    constraints = [x**2 + y**2 - 1]
    solve_lagrange(expression, variables, constraints,
                global_min_exist=True,
                global_max_exist=True)
```

Функция для задания данных по варианту 4.

Пример работы

Проверим работу на данных варианта 4.

Начальные данные:

```
expression = x**2 + 2 * y**2
constraints = [x**2 + y**2 - 1]
```

Вывод программы:

```
Целевая функция: f = x**2 + 2*y**2
Ограничения (нулевая форма):
x**2 + y**2 - 1
Функция Лагранжа: L = lambda1*(x**2 + y**2 - 1) + x**2 + 2*y**2
Уравнения стационарности и ограничения:
2*lambda1*x + 2*x = 0
2*lambda1*y + 4*y = 0
x**2 + y**2 - 1 = 0
```

Рисунок 1 - Вывод программы, часть 1

```
Кандидат 1:
  точка: [-1, 0]
  множители: [-1]
  значение функции: 1
  невязки ограничений: [0]
  получена: при решении системы
  классификация: глобальный минимум
```

Рисунок 2 - Вывод программы, часть 2

```
Кандидат 2:
  точка: [0, -1]
  множители: [-2]
  значение функции: 2
  невязки ограничений: [0]
  получена: при решении системы
  классификация: глобальный максимум
```

Рисунок 3 - Вывод программы, часть 3

```
Кандидат 3:
  точка: [0, 1]
  множители: [-2]
  значение функции: 2
  невязки ограничений: [0]
  получена: при решении системы
  классификация: глобальный максимум
```

Рисунок 4 - Вывод программы, часть 4

```
Кандидат 4:
  точка: [1, 0]
  множители: [-1]
  значение функции: 1
  невязки ограничений: [0]
  получена: при решении системы
  классификация: глобальный минимум
```

Рисунок 5 - Вывод программы, часть 5

```
Итог:
  глобальный минимум: 1
  точки минимума: [(-1, 0), (1, 0)]
  глобальный максимум: 2
  точки максимума: [(0, -1), (0, 1)]
-----------------------------------------
Количество найденных кандидатов: 4
Система стационарности решена полностью: да
Поиск особых точек выполнен полностью: да
```

Рисунок 6 – Вывод программы, часть 6

Сравнение результатов

Характеристика	Ручное решение	Программное решение
Точки-кандидаты	(1, 0), (−1, 0), (0, 1), (0, −1)	(1, 0), (−1, 0), (0, 1), (0, −1)
Множители	-1, -1, -2, -2	-1, -1, -2, -2
Значения функции	1, 1, 2, 2	1, 1, 2, 2
Минимум	1	1
Максимум	2	2

Получаем, что значения при ручном решении и программном решении совпадают, что косвенно подтверждает правильность решения и программы.

Вывод

В ходе лабораторной работы рассмотрена задача условной оптимизации для варианта 4: найти глобальные минимум и максимум функции $f(x, y) = x^2 + 2y^2$ при ограничении $x^2 + y^2 = 1$.

Проверено, что целевая функция и ограничение непрерывно дифференцируемы, допустимое множество непусто, а градиент ограничения не обращается в ноль на допустимом множестве. Следовательно, метод множителей Лагранжа применим.

Допустимое множество является замкнутым и ограниченным, то есть компактным. Поскольку целевая функция непрерывна, по теореме Вейерштрасса глобальные минимум и максимум существуют.

Построена функция Лагранжа и получена система стационарности. При ручном решении найдены четыре точки-кандидата: (1, 0), (−1, 0), (0, 1) и (0, −1). Все точки удовлетворяют исходному ограничению.

Вычисление значений целевой функции показало, что глобальный минимум равен 1 и достигается в точках (1, 0) и (−1, 0), а глобальный максимум равен 2 и достигается в точках (0, 1) и (0, −1).

Программное решение подтвердило найденные значения.

Ограничения моего программного решения:

1) Невозможность задать неравенство в начальных ограничениях,
2) Отсутствие программного способа доказать наличие глобальных экстремумов, только через изначальное задание этих параметров,
3) Отсутствие распределения кандидатов на локальные минимумы и максимумы.